

What are Document Loaders?

Document Loader is one of the components of the LangChain framework. It is responsible for loading documents from different sources. The documents are loaded in the form of Document objects that can be easily used by other components like LLMs, Text Splitters, Vector Stores, etc.

The data sources could be PDFs, Word Files, Web Pages, CSV Files, etc. There is a separate class to load data from these documents.

Document loaders play a foundational role in Retrieval-Augmented Generation (RAG). In RAG, we provide LLM with external data source to produce responses. Document loaders help in loading data into a standardized format Document object. We will be studying RAG concepts in the upcoming articles of this series, once we are familiar with all components of LangChain.

Let us understand a few important Document Loaders.

Types of Document Loaders

Depending upon the types of data sources, we have different classes to load documents. For example PDF, word, CSV files, web pages, etc. But these classes share a common working pattern.

LangChain operates on Document objects, which are simple data structures containing:

page\_content: The text content of the document.

metadata: Optional metadata such as file name, source, page number, etc.

Text Loader

TextLoader class is used to load text files with plain text. These files have .txt extension. Below is an example of how to load a text file using TextLoader

```
from langchain_community.document_loaders import TextLoader
```

```
loader = TextLoader("AI_Introduction.txt")
```

```
documents = loader.load()
```

```
print(f"Type of Documents: {type(documents)}")
```

```
print(f"Type of Document in Documents: {type(documents[0])}")
```

```
print(f"Number of Documents: {len(documents)}")
```

```
print(f"Actual Data:
```

```
{documents[0].page_content}
```

```
")
```

```
print(f"Meta Data:
```

```
{documents[0].metadata}
```

```
")
```

Press enter or click to view image in full size

### Output of TextLoader

First we create an object of TextLoader classes, passing the path of text file. We call .load method on this object that returns list of document objects. The number of Document objects created depends on the amount of the data file containing. Here, since data is less only one document object is present in the list.

Each Document object consists of actual data in page\_content and metadata in metadata .

Similarly other data loaders work, only the class and source type changes.

### PDF Loader

This is used to load content from .pdf files. There are different classes of pdf loader depending on the data present in the file.

Simple, clean PDFs: Use PyPDFLoader

PDFs with tables/columns: Use PDFPlumberLoader

Scanned/image PDFs: Use UnstructuredPDFLoader or AmazonTextractPDFLoader

Need layout and image data: Use PyMuPDFLoader

Want best structure extraction: Use UnstructuredPDFLoader

We will use PyPDFLoader in the example below.

```
from langchain_community.document_loaders import PyPDFLoader
```

```
loader = PyPDFLoader("AI_Introduction.pdf")
```

```
documents = loader.load()
```

```
print(f"Actual Data:
{documents[0].page_content}
")
print(f"Meta Data:
{documents[0].metadata}
")
```

Press enter or click to view image in full size

## Output of PDF Loader

### Web Page Loader

Web Page Loader is used to load content from the website. It uses requests and BeautifulSoup4 python libraries in the backend. We simply need to pass the URL of the website.

Get Mangesh Salunke's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

In the example below, I am loading this website:

<https://medium.com/@mangeshsalunke1309/list/generative-ai-langchain-fca1591f6e98>

```
from langchain_community.document_loaders import WebBaseLoader
```

```
URL = "https://medium.com/@mangeshsalunke1309/list/generative-ai-langchain-fca1591f6e98"
```

```
loader = WebBaseLoader(URL)
```

```
documents = loader.load()
```

```
# print(documents)
```

```
print(f"Actual Data:
```

```
{documents[0].page_content}")
```

```
print(f"Meta Data:
```

```
{documents[0].metadata}
```

")

Press enter or click to view image in full size

### Output of Web Page Loader

Visit the URL above and see the content on it. You will realize the following limitations of WebBaseLoader class:

Only static pages are loaded. (Since BeautifulSoup is used). JavaScript-rendered content is not loaded.

HTML elements like tables, headings, lists, and sections are often flattened into plain text. See in the output image, the output is a simple plain text.

You can read about other Web Page Loader classes to experiment with other ways.

### Directory Loader

Directory loader helps in loading data from multiple files stored in a specific directory.

DirectoryLoader loads files from a folder using UnstructuredLoader by default, which supports formats like PDF, HTML, and Markdown. We can use the glob parameter to include specific file types—e.g., load only .pdf files while skipping .csv and .txt.

```
from langchain_community.document_loaders import DirectoryLoader
```

```
from langchain_community.document_loaders import PyPDFLoader
```

```
loader = DirectoryLoader("Data Folder/", glob="**/*.pdf", loader_cls=PyPDFLoader,  
show_progress=True)
```

```
documents = loader.load()
```

```
print(f"Number of Documents: {len(documents)}")
```

```
for idx, document in enumerate(documents, start=1):
```

```
    print(f"  
Document {idx}")
```

```
    print("Content:  
", document.page_content)
```

```
    print("Metadata:  
", document.metadata)
```

## Output of Document Loader

The above code loads documents in the directory Data Folder .

## Directory Structure

### Conclusion

These were a few commonly used document loaders widely used. There are others that are rarely used. You can go through the documentation to find what Document Loader meets your requirements.

Relevant Articles to Read (click titles to read):

[Runnables in LangChain](#)

[Simple and Sequential Chains in LangChain](#)

[Parallel and Conditional Chains in LangChain](#)

[Output Parsers in LangChain](#)

[Getting Structured Outputs from LLM](#)

[Prompts in LangChain with Examples](#)

[Interacting with LLMs using LangChain](#)

[Basics of LangChain with Codes](#)